

# C++ Interview Test Bank

---

## Constructors, Destructors, Copy/Move, const/static/constexpr, Virtual/Polymorphism, Casts, References

Այս ֆայլը միայն այն թեմաներից է, որ արդեն ուսումնասիրել ենք: Երեք մաս.

Մաս A — 50 multiple-choice թեստ (4 տարբերակ, մեկը ճիշտ), կոդի օրինակներով՝ ինչ կտաի, error է թե ոչ, ինչ error:

Մաս B — 50 բաց (open) interview հարց՝ առանց տարբերակների:

Մաս C — 100 tricky թեստ. overload resolution (`int`, `int&`, `const int&`, `int&&`, `const int&&` — դր ֆունկցիան կկանչվի), ambiguity, compile error vs undefined behavior, sequence points (`foo(x++, x++)` տիպի), slicing, const/ref overloads, smart pointers և այլն:

Բոլոր պատասխանները **միայն վերջի էջերում** են (Answer Key A, B, C): Առաջ նայելուց առաջ փորձիր ինքդ պատասխանել:

---

# ՄԱՍ A — Multiple Choice (50)

---

## Q1

```
class A { public: A() { cout << "A"; } };  
int main() { A a; }
```

Ի՞նչ կտպի: - (a) ոչինչ - (b) A - (c) compile error - (d) AA

## Q2

Constructor-ի return type-ը ի՞նչ է: - (a) void - (b) int - (c) նույն class-ի type-ը - (d) չունի return type

## Q3

```
class A { public: void A() {} };
```

Ի՞նչ է սա: - (a) նորմալ constructor - (b) compile error (constructor-ը return type չունի) - (c) destructor - (d) copy constructor

## Q4

```
class A {  
public:  
    A() { cout << "default "; }  
    A(int x) { cout << "param "; }  
};  
int main() { A a; A b(5); }
```

Ի՞նչ կտպի: - (a) default param - (b) param default - (c) default default - (d) param param

## Q5

Ո՞ր constructor-ն է կանչվում `A b = a;` (a-ն արդեն A object է): - (a) default constructor - (b) parameterized constructor - (c) copy constructor - (d) move constructor

## Q6

```
class A { A() {} };  
int main() { A a; }
```

Ի՞նչ կլինի: - (a) OK - (b) compile error (constructor-ը private է) - (c) runtime error - (d) տպում է A

## Q7

Ի՞նչ է Initialization List-ը `A(int n) : x(n) {}` -ում: - (a) x-ը assign է անում մարմնում - (b) x-ը initialize է անում մարմնից առաջ - (c) նոր function է - (d) compile error

## Q8

Constructor-ը կարող է virtual լինել: - (a) այո - (b) ոչ, քանի որ object-ը դեռ գոյություն չունի - (c) միայն abstract class-ում - (d) միայն C++11-ից

## Q9

Destructor-ի syntax-ը ո՞րն է: - (a) `!ClassName()` - (b) `~ClassName()` - (c) `delete ClassName()` - (d) `ClassName::~~()`

## Q10

```
{ Test t; } cout << "end";
```

Test-ը ունի destructor, որ տպում է "D": Ի՞նչ հերթականությամբ: - (a) end D - (b) D end - (c) միայն end - (d) միայն D

## Q11

Heap object-ի destructor-ը երբ է կանչվում:

```
Test* p = new Test();
```

- (a) scope-ից դուրս ավտոմատ
- (b) երբ `delete p;` անես
- (c) երբեք
- (d) program start-ին

## Q12

Destructor-ը կարող է parameter ունենալ: - (a) այո - (b) ոչ - (c) միայն int - (d) միայն default

## Q13

Քանի՞ destructor կարող է ունենալ class-ը: - (a) անսահման - (b) 2 - (c) 1 - (d) 0

## Q14

```
Test a; Test b;
```

LIFO-ի դեպքում որ destructor-ն է առաջինը կանչվում: - (a) a-ինը - (b) b-ինը - (c) միաժամանակ - (d) ոչ մեկը

## Q15

```
class Base { public: ~Base(){cout<<"B";} };  
class Derived: public Base { public: ~Derived(){cout<<"D";} };  
Base* p = new Derived();  
delete p;
```

Ի՞նչ կտպի (destructor-ը virtual Չէ): - (a) DB - (b) BD - (c) B - (d) D

## Q16

Նույն Q15-ը, բայց `virtual ~Base()`: Ի՞նչ կտպի: - (a) B - (b) D - (c) DB - (d) BD

## Q17

Ինչո՞ւ է virtual destructor-ը կարևոր: - (a) արագացնում է կոդը - (b) որ base pointer-ով delete-ի ժամանակ derived destructor-ն էլ կանչվի - (c) memory խնայում է - (d) պարտադիր է ամեն class-ում

## Q18

```
class Test { int* data; public:  
    Test(){ data=new int(5); }  
    ~Test(){ delete data; }  
};  
Test a; Test b = a;
```

Ի՞նչ խնդիր կա (default copy): - (a) ոչ մի - (b) shallow copy -> double delete - (c) compile error - (d) memory քիչ է

## Q19

Deep copy-ի ճիշտ copy constructor-ը ճիշտ է: - (a) `Test(const Test& o){ data=o.data; }` - (b) `Test(const Test& o){ data=new int(*o.data); }` - (c) `Test(Test o){ data=o.data; }` - (d) `Test(){ data=new int; }`

## Q20

Ինչո՞ւ copy constructor-ը `const Test&` է ընդունում, ոչ `Test`: - (a) գեղեցիկ է - (b) որ copy չլինի և infinite recursion-ից խուսափի - (c) compile error է լինում այլապես միշտ - (d) տարբերություն չկա

## Q21

```
void f(std::string& s){}
f("Arman");
```

Ի՞նչ կլինի: - (a) OK - (b) compile error (non-const ref-ը temporary չի ընդունում) - (c) runtime error - (d) տպում է Arman

## Q22

```
void f(const std::string& s){}
f("Arman");
```

Ի՞նչ կլինի: - (a) OK - (b) compile error - (c) runtime error - (d) memory leak

## Q23

```
void f(std::string&& s){}
f("Arman");
```

Ի՞նչ կլինի: - (a) compile error - (b) OK (rvalue reference-ը temporary-ն ընդունում է) - (c) runtime error - (d) infinite loop

## Q24

`std::move(x)` իրականում ի՞նչ է անում: - (a) տեղափոխում է x-ի տվյալները - (b) ջնջում է x-ը - (c) cast է անում rvalue-ի (`static_cast<T&&>`) - (d) copy է անում

## Q25

```
int&& r = 5;
```

`r`-ը հետո օգտագործելիս ի՞նչ է: - (a) rvalue - (b) lvalue (անուն ունի) - (c) pointer - (d) const

## Q26

```
int* a = new int;  
cout << *a;
```

Ի՞նչ կտպի: - (a) 0 - (b) undefined (աղբ արժեք) - (c) compile error - (d) nullptr

## Q27

```
int* b = new int();  
cout << *b;
```

Ի՞նչ կտպի: - (a) undefined - (b) 0 - (c) compile error - (d) 1

## Q28

`std::string_view`-ը ի՞նչ է պահում: - (a) string-ի ամբողջ copy - (b) pointer + length - (c) միայն length - (d) heap allocation

## Q29

SSO (Small String Optimization) ի՞նչ է անում: - (a) փոքր string-ը պահում է object-ի ներսում, heap չի գնում - (b) string-ը compress է անում - (c) միշտ heap է օգտագործում - (d) string-ը ջնջում է

## Q30

```
const int x = 10;  
x = 20;
```

Ի՞նչ կլինի: - (a) OK, x=20 - (b) compile error (x-ը const է) - (c) runtime error - (d) x=10 մնում է լուռ

## Q31

```
const int* p;
```

Ի՞նչն է const: - (a) pointer-ը - (b) տվյալը (որին ցույց է տալիս) - (c) երկուսն էլ - (d) ոչ մեկը

### Q32

```
int* const p = &x;
```

Ի՞նչն է const: - (a) տվյալը - (b) pointer-ը - (c) երկուսն էլ - (d) ոչ մեկը

### Q33

```
void foo(){ static int x=0; x++; cout<<x; }  
foo(); foo(); foo();
```

Ի՞նչ կտպի: - (a) 111 - (b) 123 - (c) 000 - (d) 333

### Q34

Static class member-ը ընդհանուր է պատկանում: - (a) յուրաքանչյուր object-ին առանձին - (b) class-ին (մեկ instance բոլորի համար) - (c) main function-ին - (d) ոչ մեկին

### Q35

```
class A { int age; public: static void f(){ cout<<age; } };
```

Ի՞նչ կլինի: - (a) OK - (b) compile error (static function-ը non-static member չի տեսնում) - (c) տպում է 0 - (d) runtime error

### Q36

const vs constexpr — ի՞նչն է ճիշտ: - (a) նույն բանն են - (b) constexpr = compile-time constant, const = immutable object - (c) const = compile-time միայն - (d) constexpr-ը runtime-ում է

### Q37

```
Animal* p = new Dog();  
p->speak();
```

speak() virtual Չէ: Base-ը տպում է "Animal", Dog-ը "Dog": Ի՞նչ կտպի: - (a) Dog - (b) Animal - (c) compile error - (d) AnimalDog

### Q38

Նույնը, բայց speak() virtual է: Ի՞նչ կտպի: - (a) Animal - (b) Dog - (c) compile error - (d) ոչինչ

### Q39

`virtual void f() = 0;` ինչ է: - (a) սովորական virtual - (b) pure virtual (class-ը abstract է դառնում) - (c) static function - (d) compile error

### Q40

```
class Shape { public: virtual double area()=0; };  
Shape s;
```

Ի՞նչ կլինի: - (a) OK - (b) compile error (abstract class-ից object չի ստեղծվում) - (c) runtime error - (d) area()=0

### Q41

vtable-ը ռմբ է պատկանում: - (a) object-ին - (b) class-ին (մեկ հատ per polymorphic class) - (c) function-ին - (d) vptr-ին

### Q42

vptr-ը որտեղ է պահվում: - (a) class-ում - (b) object-ի ներսում - (c) stack-ում առանձին - (d) vtable-ում

### Q43

```
class Base { public: virtual void f(int){} };  
class Derived: public Base { public: void f(double){} };
```

Derived-ի `f(double)` -ը ինչ է Base-ի նկատմամբ: - (a) override - (b) overload - (c) ոչ override, ոչ overload (նոր function, name hiding) - (d) compile error

### Q44

`override` keyword-ը ինչի համար է: - (a) արագացնում է - (b) compiler-ը բռնում է սխալ signature-ը - (c) պարտադիր է virtual-ի հետ - (d) memory խնայում է

### Q45

```
class A {};  
A* a = new B();  
B* b = dynamic_cast<B*>(a);
```

*// ոչ polymorphic*  
*// B: public A*



Ի՞նչ կլինի: - (a) OK - (b) compile error ('A' is not polymorphic) - (c) nullptr - (d) valid pointer

## Q46

```
Base* p = new Cat();      // Cat: public Animal, polymorphic
Dog* d = dynamic_cast<Dog*>(p);
```

**d** -ն ի՞նչ է: - (a) valid Dog\* - (b) nullptr - (c) compile error - (d) undefined

## Q47

Reference-ի dynamic\_cast ձախողվելիս ի՞նչ է լինում: - (a) nullptr - (b) throw std::bad\_cast - (c) compile error - (d) 0

## Q48

```
const int y = 5;
int* q = const_cast<int*>(&y);
*q = 10;
```

Ի՞նչ է սա: - (a) OK, y=10 - (b) compile error - (c) undefined behavior (իսկապես const object-ի փոփոխում) - (d) y=5 երաշխավորված

## Q49

C-style **(T)x** cast-ը ի՞նչ հերթականությամբ է compiler-ը փորձում: - (a) միայն static\_cast - (b) const -> static -> static+const -> reinterpret -> reinterpret+const - (c) միայն reinterpret\_cast - (d) dynamic\_cast առաջինը

## Q50

```
Base* b = new Base();
Derived* d = static_cast<Derived*>(b); // downcast, b-ն իրականում Base է
```

Ի՞նչ է սա: - (a) compile error - (b) nullptr - (c) կոմպիլվում է, բայց undefined behavior (static\_cast չի ստուգում) - (d) valid pointer

## ՄԱՍ B — Open Questions (50)

---

Սրանք բաց հարցեր են (առանց տարբերակների): Փորձիր բարձրաձայն, ամբողջական պատասխանել, ասես interview-ում ես: Մոդելային պատասխանները՝ Answer Key B-ում:

1. Ի՞նչ է constructor-ը և ինչու է պետք:
2. Constructor-ը return type ունի, և ինչու:
3. Ի՞նչ է default constructor-ը:
4. Ի՞նչ է parameterized constructor-ը:
5. Ի՞նչ է constructor overloading-ը:
6. Ի՞նչ է initialization list-ը և ինչու է assignment-ից լավ:
7. Constructor-ը կարող է virtual լինել: Ինչու:
8. Constructor-ը կարող է private լինել: Որտեղ է դա օգտագործվում:
9. Բացատրիր Singleton pattern-ը և ինչու է constructor-ը private:
10. Ի՞նչ է destructor-ը և երբ է կանչվում:
11. Stack և heap object-ների destructor-ը երբ է կանչվում:
12. Ինչու է destructor-ը պետք (resource management):
13. Destructor-ը parameter կամ return type ունի:
14. Կարող է class-ը մի քանի destructor ունենալ:
15. Ի՞նչ է destructor call order-ը (LIFO):
16. Ի՞նչ է virtual destructor-ը և ինչու է կարևոր:
17. Ի՞նչ է լինում, եթե base destructor-ը virtual չէ և delete անես base pointer-ով:
18. Ի՞նչ է copy constructor-ը:
19. Երբ է copy constructor-ը կանչվում (3 դեպք):
20. Ի՞նչ է shallow copy-ն:
21. Ի՞նչ է deep copy-ն:
22. Ինչո՞ւ copy constructor-ը `const&` -ով է ընդունում argument-ը:
23. Ի՞նչ է double delete-ը և ինչպես է առաջանում:
24. Ի՞նչ է move constructor-ը և ինչու է copy-ից արագ:
25. Ի՞նչ է rvalue reference-ը (`&&`):
26. `std::move` -ը իրականում ի՞նչ է անում:
27. Բացատրիր lvalue և rvalue-ի տարբերությունը:
28. Ինչու է `int&& r = 5;` -ից հետո `r` -ը lvalue:
29. Ի՞նչ է Rule of Zero-ն:
30. `new int` vs `new int()` — ի՞նչ տարբերություն:
31. Ի՞նչ է temporary object-ը:
32. Ի՞նչ է `std::string_view` -ը և ինչու է օգտիմալ:
33. Ի՞նչ են RVO և NRVO-ն:
34. Ի՞նչ է SSO-ն:

35. Ի՞նչ է `const`-ը և ինչ է երաշխավորում:
  36. `const int*` vs `int* const` — բացատրիր:
  37. Ի՞նչ է const member function-ը:
  38. Ինչու `const string&` parameter — 2 պատճառ:
  39. Ի՞նչ է static local variable-ը և երբ է ստեղծվում/մահանում:
  40. Static class member-ը ի՞նչ է պատկանում և ինչպես ես define անում:
  41. Ինչու static member function-ը չի կարող non-static member օգտագործել:
  42. `const` vs `constexpr` — տարբերությունը:
  43. Ի՞նչ է virtual function-ը և runtime polymorphism-ը:
  44. Static binding vs dynamic binding — բացատրիր:
  45. Override vs overload — մանրամասն տարբերությունը:
  46. Ի՞նչ է vtable-ը և vptr-ը, ի՞նչ level-ում են:
  47. Քայլ առ քայլ ի՞նչ է կատարվում virtual call-ի ժամանակ:
  48. Ի՞նչ է pure virtual function-ը և abstract class-ը:
  49. Բացատրիր 4 C++ cast-երը և երբ որն օգտագործել:
  50. static\_cast vs dynamic\_cast downcast-ի ժամանակ — տարբերությունը:
- 
-

# ՄԱՍ C — Tricky / Overload Resolution / UB (100)

Այս մասը ամենակարևորն է հարցազրույցի համար. «ո՞ր overload-ը կկանչվի», «ambiguous է թե ոչ», «compile error թե undefined behavior», value category-ներ (lvalue/rvalue), reference binding ( `int` , `int&` , `const int&` , `int&&` , `const int&&` ), conversions, slicing, sequence points: Բոլոր պատասխանները՝ **Answer Key C**-ում (վերջում):

Ամեն հարցում ֆունկցիաները տպում են label (օր. `lref` , `clref` , `rref` , `crref` , `val` ):

## C1

```
void f(int&){cout<<"lref";}
void f(const int&){cout<<"clref";}
void f(int&&){cout<<"rref";}
int x = 5;
f(x);
```

- (a) lref
- (b) clref
- (c) rref
- (d) ambiguous (CE)

## C2

Նույն overload set (C1):

```
f(5);
```

- (a) lref
- (b) clref
- (c) rref
- (d) CE

## C3

Նույն set (C1):

```
const int cx = 5;  
f(cx);
```

- (a) lref
- (b) clref
- (c) rref
- (d) CE

## C4

ზოგადი set (C1):

```
int x = 5;  
f(std::move(x));
```

- (a) lref
- (b) clref
- (c) rref
- (d) CE

## C5

ზოგადი set (C1):

```
int x = 5;  
f(x + 1);
```

- (a) lref
- (b) clref
- (c) rref
- (d) CE

## C6

ზოგადი set (C1):

```
const int cx = 5;  
f(std::move(cx)); // const xvalue
```

- (a) lref
- (b) clref

- (c) rref
- (d) CE

## C7

```
void f(int&){cout<<"lref";}  
void f(const int&){cout<<"clref";}  
f(5);
```

- (a) lref
- (b) clref
- (c) rref
- (d) CE

## C8

ئۇنىۋېرستېت (C7):

```
int x=5; f(x);
```

- (a) lref
- (b) clref
- (c) CE
- (d) rref

## C9

```
void f(int){cout<<"val";}  
void f(int&){cout<<"lref";}  
int x=5; f(x);
```

- (a) val
- (b) lref
- (c) ambiguous (CE)
- (d) rref

## C10

ئۇنىۋېرستېت (C9):

```
f(5);
```

- (a) val
- (b) lref
- (c) CE
- (d) rref

## C11

```
void f(int){cout<<"val";}  
void f(int&&){cout<<"rref";}  
f(5);
```

- (a) val
- (b) rref
- (c) ambiguous (CE)
- (d) clref

## C12

හරියු set (C11):

```
int x=5; f(x);
```

- (a) val
- (b) rref
- (c) CE
- (d) ambiguous

## C13

```
void f(const int&){cout<<"clref";}  
void f(const int&&){cout<<"crref";}  
f(5);
```

- (a) clref
- (b) crref
- (c) CE
- (d) ambiguous

## C14

හරියු set (C13):

```
int x=5; f(x);
```

- (a) clref
- (b) cref
- (c) CE
- (d) ambiguous

## C15

Նույն set (C13):

```
int x=5; f(std::move(x));
```

- (a) clref
- (b) cref
- (c) CE
- (d) val

## C16

```
void f(int&){cout<<"lref";}  
void f(int&&){cout<<"rref";}  
const int cx=5; f(cx);
```

- (a) lref
- (b) rref
- (c) CE (const lvalue չի bind լինում ոչ մեկին)
- (d) clref

## C17

```
void f(int&){cout<<"lref";}  
void f(int&&){cout<<"rref";}  
f(5);
```

- (a) lref
- (b) rref
- (c) CE
- (d) ambiguous



## C18

```
void f(char){cout<<"char";}  
void f(int){cout<<"int";}  
f('A');
```

- (a) char
- (b) int
- (c) ambiguous
- (d) CE

## C19

Use set (C18):

```
f(true);
```

- (a) char
- (b) int
- (c) ambiguous
- (d) CE

## C20

```
void f(int){cout<<"int";}  
void f(double){cout<<"double";}  
f(3.14f); // float
```

- (a) int
- (b) double
- (c) ambiguous
- (d) CE

## C21

```
void f(long){cout<<"long";}  
void f(double){cout<<"double";}  
f(5); // int
```

- (a) long
- (b) double

- (c) ambiguous (CE)
- (d) int

## C22

```
void f(unsigned){cout<<"u";}  
void f(long){cout<<"long";}  
f(5);    // int
```

- (a) u
- (b) long
- (c) ambiguous (CE)
- (d) int

## C23

```
void f(int){cout<<"int";}  
void f(char*){cout<<"ptr";}  
f(0);
```

- (a) int
- (b) ptr
- (c) ambiguous
- (d) CE

## C24

İnci set (C23):

```
f(nullptr);
```

- (a) int
- (b) ptr
- (c) ambiguous
- (d) CE

## C25

```
void f(const std::string&){cout<<"str";}
void f(const char*){cout<<"cstr";}
f("hello");
```

- (a) str
- (b) cstr
- (c) ambiguous
- (d) CE

## C26

```
void f(int&){cout<<"lref";}
int x=5;
const int cx=10;
f(cx);
```

- (a) lref
- (b) clref
- (c) CE (int& չի ընդունում const)
- (d) val

## C27

```
struct S { void g(){cout<<"nc";} void g() const {cout<<"c";} };
S s; s.g();
```

- (a) nc
- (b) c
- (c) ambiguous
- (d) CE

## C28

Նույն (C27):

```
const S s; s.g();
```

- (a) nc
- (b) c
- (c) ambiguous

- (d) CE

## C29

```
struct S {
    void g() & { cout<<"lval"; }
    void g() && { cout<<"rval"; }
};
S s; s.g();
```

- (a) lval
- (b) rval
- (c) ambiguous
- (d) CE

## C30

Նույն (C29):

```
S{}.g();
```

- (a) lval
- (b) rval
- (c) ambiguous
- (d) CE

## C31

```
int x = 9;
foo(x++, x++);    // foo(int,int)
```

Ի՞նչ է սա C++17-ում: - (a) երաշխավորված foo(9,10) - (b) unspecified order (որ argument-ը 9 կստանա անհայտ է), բայց ոչ UB - (c) միշտ UB - (d) compile error

## C32

```
int i = 0;
i = i++;    // C++17
```

- (a) i == 1
- (b) i == 0 (C++17-ում defined)

- (c) UB
- (d) CE

## C33

```
int i = 0;
i = ++i + i++;
```

- (a) defined, i==2
- (b) UB (unų object-ի unsequenced modifications)
- (c) i==1
- (d) CE

## C34

```
int a[3] = {0,1,2};
int i = 0;
a[i] = i++;    // C++17
```

- (a) UB
- (b) C++17-ում defined (i++ sequenced before LHS index)
- (c) CE
- (d) սիշտո crash

## C35

```
int x = 5;
int y = x++ + ++x;
```

- (a) defined, y==12
- (b) UB (x-ի unsequenced modifications)
- (c) y==11
- (d) CE

## C36

```

class Base { public: virtual void f(){cout<<"B";} };
class Derived: public Base { public: void f(){cout<<"D";} };
Derived d;
Base b = d;    // slicing
b.f();

```

- (a) D
- (b) B (object slicing)
- (c) CE
- (d) undefined

## C37

```

class Base { public: virtual void f(){cout<<"B";} };
class Derived: public Base { public: void f() override {cout<<"D";} };
Derived d;
Base& b = d;
b.f();

```

- (a) B
- (b) D
- (c) CE
- (d) undefined

## C38

```

const std::string s = "hi";
std::string s2 = std::move(s);    // s-ը const է

```

- (a) move է լինում
- (b) copy է լինում (const-ը move չի լինում, copy constructor է ընտրվում)
- (c) CE
- (d) UB

## C39

```

void f(int&&){cout<<"rref";}
int&& r = 5;
f(r);    // r-ը անուշի ունի

```

- (a) rref
- (b) CE (r-ը lvalue է, int&&-ին չի bind լինում)
- (c) val
- (d) lref

## C40

Նույն (C39), բայց`

```
f(std::move(r));
```

- (a) rref
- (b) CE
- (c) lref
- (d) UB

## C41

```
void f(int&){cout<<"lref";}
void f(const int&){cout<<"clref";}
void f(int&&){cout<<"rref";}
void f(const int&&){cout<<"crref";}
f(5);
```

- (a) rref
- (b) crref
- (c) clref
- (d) ambiguous

## C42

Նույն 4-overload set (C41):

```
const int cx=5; f(std::move(cx));
```

- (a) rref
- (b) crref
- (c) clref
- (d) CE

## C43

Նույն set (C41):

```
int x=5; f(x);
```

- (a) lref
- (b) clref
- (c) rref
- (d) crref

## C44

```
template<class T> void f(T&&){cout<<"universal";}
int x=5; f(x);
```

T&& -ը forwarding reference է: Ի՞նչ կլինի: - (a) CE (lvalue չի bind լինում) - (b) universal (T = int&, forwarding ref-ը lvalue-ն էլ է ընդունում) - (c) ambiguous - (d) rref

## C45

```
void f(bool){cout<<"bool";}
void f(const std::string&){cout<<"str";}
f("hello"); // string literal
```

- (a) str
- (b) bool (const char\* -> bool conversion)
- (c) ambiguous
- (d) CE

## C46

```
void f(int){cout<<"int";}
void f(long){cout<<"long";}
f(5L);
```

- (a) int
- (b) long
- (c) ambiguous
- (d) CE

## C47



```
short s = 5;
void f(int){cout<<"int";}
void f(short){cout<<"short";}
f(s);
```

- (a) int
- (b) short
- (c) ambiguous
- (d) CE

## C48

```
void f(int){cout<<"int";}
void f(short){cout<<"short";}
f(5);    // int literal
```

- (a) int
- (b) short
- (c) ambiguous
- (d) CE

## C49

```
enum Color { RED };
void f(int){cout<<"int";}
f(RED);
```

- (a) int (unscoped enum -> int)
- (b) CE
- (c) ambiguous
- (d) undefined

## C50

```
enum class Color { RED };
void f(int){cout<<"int";}
f(Color::RED);
```

- (a) int
- (b) CE (scoped enum-ը implicit չի փոխարկվում int-ի)

- (c) ambiguous
- (d) RED

## C51

```
void f(int*){cout<<"ptr";}
void f(int){cout<<"int";}
f(NULL);
```

- (a) ptr
- (b) int (NULL հաճախ 0 -> int)
- (c) ambiguous
- (d) CE

## C52

```
struct A { A(int){} };
void f(A){cout<<"A";}
void f(double){cout<<"double";}
f(5);    // int
```

- (a) A (user-defined conversion)
- (b) double (standard conversion preferred)
- (c) ambiguous
- (d) CE

## C53

```
void f(int){cout<<"a";}
void f(int){cout<<"b";}

```

- (a) OK
- (b) CE (redefinition, նույն signature)
- (c) overload
- (d) runtime error

## C54

```
int f(){return 1;}
double f(){return 2.0;}
```

- (a) overload by return type
- (b) CE (return type-ը overload չի սահմանում)
- (c) OK
- (d) ambiguous

## C55

```
void f(int x){}
void f(int x, int y=5){}
f(3);
```

- (a) OK, կանչում է առաջինը
- (b) ambiguous (CE)
- (c) կանչում է երկրորդը
- (d) OK, երկրորդը

## C56

```
class A {
public:
    A(){cout<<"ctor ";}
    A(const A&){cout<<"copy ";}
    A(A&&){cout<<"move ";}
};
A a;
A b = std::move(a);
```

- (a) ctor copy
- (b) ctor move
- (c) ctor ctor
- (d) move ctor

## C57

Նույն (C56):

```
A a;
A b = a;
```

- (a) ctor copy
- (b) ctor move

- (c) ctor
- (d) copy

## C58

```
A make(){ A a; return a; } // A ունի ctor/copy/move տարբերակներ
A x = make();
```

C++17-ում ինչ կտալի (guaranteed copy elision / NRVO): - (a) ctor (սխալ, RVO-ով copy/move վերացվում է; NRVO-ն շատ դեպքում) - (b) ctor copy - (c) ctor move move - (d) copy copy

## C59

```
void f(std::string s){}
std::string a="hi";
f(std::move(a));
```

- (a) copy constructor
- (b) move constructor
- (c) CE
- (d) ոչինչ

## C60

```
int x = 5;
auto&& r = x; // forwarding-like
```

**r** -ի տիպն ինչ է: - (a) int&& - (b) int& (lvalue -> auto&& = int&) - (c) int - (d) CE

## C61

```
int x = 5;
auto&& r = 10;
```

**r** -ի տիպն ինչ է: - (a) int&& - (b) int& - (c) int - (d) CE

## C62

```
const int x = 5;
auto y = x;          // top-level const dropped
y = 10;
```

- (a) CE (y-ը const է)
- (b) OK (y-ը int է, const չի պահպանվում)
- (c) UB
- (d) x=10

## C63

```
int arr[5];
cout << sizeof(arr) / sizeof(arr[0]);
```

- (a) 5
- (b) 1
- (c) 20
- (d) CE

## C64

```
void f(int arr[5]){ cout << sizeof(arr); } // 64-bit
```

- (a) 20
- (b) 8 (array -> pointer decay)
- (c) 5
- (d) CE

## C65

```
class A { public: int x; };
A a;
cout << a.x;
```

- (a) 0
- (b) undefined (uninitialized)
- (c) CE
- (d) garbage միշտ 0

## C66

```
int x = 10;
int& r = x;
int& r2 = r;
r2 = 20;
cout << x;
```

- (a) 10
- (b) 20
- (c) CE
- (d) undefined

## C67

```
int& getRef(){ int x = 5; return x; }
int y = getRef();
```

- (a) y==5 միշտ
- (b) UB (dangling reference to local)
- (c) CE
- (d) y==0

## C68

```
const int& r = 5;    // temporary
cout << r;
```

- (a) UB
- (b) 5 (const ref-ը երկարացնում է temporary-ի lifetime)
- (c) CE
- (d) garbage

## C69

```
int&& r = 5;
r = 10;
cout << r;
```

- (a) CE

- (b) 10 (rvalue ref-ը modifiable է)
- (c) UB
- (d) 5

## C70

```
std::vector<int> v = {1,2,3};
for (auto x : v) x *= 2;
cout << v[0];
```

- (a) 2
- (b) 1 (copy by value, v չի փոխվում)
- (c) CE
- (d) UB

## C71

Նույն (C70), բայց `for (auto& x : v) x *= 2;`

```
cout << v[0];
```

- (a) 1
- (b) 2
- (c) CE
- (d) UB

## C72

```
class Base { public: ~Base(){cout<<"B";} }; // ոչ virtual
class Derived: public Base { int* p; public:
    Derived(){p=new int[100];}
    ~Derived(){ delete[] p; cout<<"D"; }
};
Base* b = new Derived();
delete b;
```

- (a) DB
- (b) B (միայն, + memory leak, non-virtual dtor)
- (c) BD
- (d) CE

## C73

```
int x = 5;  
double d = 3.9;  
cout << (x + d);
```

- (a) 8
- (b) 8.9 (int -> double promotion)
- (c) CE
- (d) 8.0

## C74

```
unsigned int a = 1;  
int b = -1;  
cout << (a > b);
```

- (a) 1 (b-ն դառնում է շատ մեծ unsigned)
- (b) 0
- (c) CE
- (d) UB

## C75

```
char c = 'A';  
cout << c << " " << (int)c;
```

- (a) A 65
- (b) 65 65
- (c) A A
- (d) CE

## C76

```
int x = 5;  
if (x = 0) cout << "yes"; else cout << "no";
```

- (a) yes
- (b) no (assignment -> 0 -> false)
- (c) CE



- (d) undefined

## C77

```
int i = 5;
cout << i++ << i;    // C++17
```

- (a) 55
- (b) 56
- (c) UB
- (d) CE (Հուշում. C++17-ում `<<`-ի ձախ operand-ը sequenced է աջից առաջ)

## C78

```
#include <memory>
std::unique_ptr<int> p1(new int(5));
std::unique_ptr<int> p2 = p1;    // copy
```

- (a) OK
- (b) CE (unique\_ptr copy չի լինում, միայն move)
- (c) runtime error
- (d) UB

## C79

Նույն (C78), բայց՝

```
std::unique_ptr<int> p2 = std::move(p1);
```

- (a) CE
- (b) OK (ownership փոխանցվում է, p1 դառնում nullptr)
- (c) double free
- (d) UB

## C80

```
std::shared_ptr<int> a = std::make_shared<int>(5);
std::shared_ptr<int> b = a;
cout << a.use_count();
```

- (a) 1
- (b) 2
- (c) CE
- (d) 0

## C81

```
void f() noexcept { throw 1; }  
f();
```

- (a) exception բռնվում է
- (b) std::terminate (noexcept function-ը throw է անում)
- (c) CE
- (d) OK

## C82

```
int x = 10;  
int y = x / 0;
```

- (a) inf
- (b) UB (integer division by zero)
- (c) 0
- (d) CE

## C83

```
double x = 10;  
double y = x / 0;
```

- (a) UB
- (b) inf (floating-point, ոչ UB շատ platform-ներում)
- (c) CE
- (d) 0

## C84

```
std::vector<int> v = {1,2,3};  
cout << v[10];
```

- (a) 0
- (b) UB (out of bounds, operator[] չի ստուգում)
- (c) CE
- (d) exception

## C85

Նույն (C84), բայց՝

```
cout << v.at(10);
```

- (a) UB
- (b) throw std::out\_of\_range
- (c) 0
- (d) CE

## C86

```
std::string s = "hello";
char c = s[10];
```

- (a) throw
- (b) UB (index out of range)
- (c) CE
- (d) '\0' միջոց

## C87

```
int* p = nullptr;
cout << *p;
```

- (a) 0
- (b) UB (null dereference)
- (c) CE
- (d) nullptr

## C88

```
int* p = new int(5);
delete p;
delete p;
```

- (a) OK
- (b) UB (double delete)
- (c) CE
- (d) leak

## C89

```
int* p = new int(5);
delete p;
cout << *p;    // use after free
```

- (a) 5 միջև
- (b) UB (dangling pointer)
- (c) CE
- (d) 0

## C90

```
class A { public: A(){ f(); } virtual void f(){cout<<"A";} };
class B: public A { public: void f() override {cout<<"B";} };
B b;
```

- (a) B
- (b) A (constructor-ի ներքոյն virtual dispatch -> ընթացիկ class)
- (c) CE
- (d) AB

## C91

```
class A { public: virtual void f(int){cout<<"A";} };
class B: public A { public: void f(int) override {cout<<"B";} };
A* a = new B();
a->f(5);
```

- (a) A
- (b) B

- (c) CE
- (d) undefined

## C92

```
class A { public: void f(){cout<<"A";} };           // ոչ virtual
class B: public A { public: void f(){cout<<"B";} };
A* a = new B();
a->f();
```

- (a) A (static binding, virtual չկա)
- (b) B
- (c) CE
- (d) undefined

## C93

```
int x = 3;
switch(x) { case 3: cout<<"3"; case 4: cout<<"4"; break; default: cout<<"d"; }
```

- (a) 3
- (b) 34 (fall-through, break չկա case 3-ից հետո)
- (c) 34d
- (d) CE

## C94

```
for (int i = 0; i < 3; i++);
    cout << "hi";
```

- (a) hihhi
- (b) hi (loop-ի մարմինը դատարկ է `;`-ի պատճառով)
- (c) CE
- (d) ոչինչ

## C95

```
int x = 5;
int y = x > 3 ? x < 10 ? 1 : 2 : 3;
cout << y;
```

- (a) 1
- (b) 2
- (c) 3
- (d) CE

## C96

```
constexpr int square(int x){ return x*x; }
int arr[square(3)];
cout << sizeof(arr)/sizeof(int);
```

- (a) 9 (constexpr-ը compile-time-ում հաշվում է)
- (b) CE
- (c) 3
- (d) UB

## C97

```
int x = 5;
const int& r = x;
x = 10;
cout << r;
```

- (a) 5
- (b) 10 (r-ը x-ի alias է, const-ը միայն r-ով փոփոխումն է արգելում)
- (c) CE
- (d) UB

## C98

```
void f(int&){cout<<"lref";}
void f(int){cout<<"val";}
int x=5;
f(std::move(x));
```

- (a) lref
- (b) val (int& չի bind լինում rvalue-ին, մնում է val)
- (c) CE
- (d) ambiguous

## C99

```
struct A { int x; };  
A a{};           // value-initialization  
cout << a.x;
```

- (a) 0 (value-init -> zero)
- (b) undefined
- (c) CE
- (d) garbage

## C100

```
struct A { int x; };  
A a;           // default-init (no {})  
cout << a.x;
```

- (a) 0
- (b) undefined (uninitialized POD member)
- (c) CE
- (d) uninitialized garbage 0

# ANSWER KEY A (Multiple Choice)

---

1-b · 2-d · 3-b · 4-a · 5-c · 6-b · 7-b · 8-b · 9-b · 10-b 11-b · 12-b · 13-c · 14-b · 15-c · 16-c · 17-b · 18-b · 19-b · 20-b 21-b · 22-a · 23-b · 24-c · 25-b · 26-b · 27-b · 28-b · 29-a · 30-b 31-b · 32-b · 33-b · 34-b · 35-b · 36-b · 37-b · 38-b · 39-b · 40-b 41-b · 42-b · 43-c · 44-b · 45-b · 46-b · 47-b · 48-c · 49-b · 50-c

## Կարճ բացատրություններ (A)

**Q1** `A a;` կանչում է constructor-ը -> "A": **Q3** `void A()` return type ունի -> constructor չէ, բայց նաև method-ը class-ի անունով հակասական է; գործնականում compile error է constructor-ի կանոնի խախտմամբ: **Q5** `A b = a;` copy initialization -> copy constructor: **Q10** Stack object-ի destructor-ը scope-ի վերջում, բայց "end"-ից հետո? Ոչ — `t` -ն ապրում է մինչ block-ի `}`; "end" տողը block-ից դուրս է, ուստի D-ն տպվում է block-ը փակվելիս, հետո... իրականում այստեղ `{Test t;}` block-ն ավարտվում է cout-ից առաջ, ուստի **D end**: **Q15** virtual չկա -> միայն Base destructor -> "B": **Q16** virtual կա -> "DB" (Derived հետո Base): **Q18** default copy -> shallow -> երկուսը նույն pointer -> double delete: **Q21** non-const lvalue ref-ը temporary չի ընդունում -> compile error: **Q24** `std::move` = `static_cast<T&&>`, ոչինչ չի տեղափոխում: **Q25** named rvalue reference -> lvalue: **Q26** `new int` uninitialized -> undefined/աղբ: **Q30** const object-ի reassignment -> compile error: **Q35** static function-ը `this` չունի -> non-static `age` չի տեսնում -> compile error: **Q37** virtual չկա -> static binding -> "Animal": **Q40** abstract class -> object չի ստեղծվում -> compile error: **Q43** signature տարբեր -> ոչ override, ոչ overload; name hiding: **Q45** `A` polymorphic չէ -> `dynamic_cast` compile error: **Q46** իրական object Cat է, ոչ Dog -> nullptr: **Q48** իսկապես const object-ի փոփոխում -> UB: **Q50** `static_cast` downcast-ը չի ստուգում -> UB:

---



# ANSWER KEY B (Open Questions)

---

1. Constructor-ը class-ի հատուկ member function է, որ ավտոմատ կանչվում է object ստեղծելիս; նպատակը՝ object-ի data member-ները initialize անել և պատրաստել օգտագործման: Անունը class-ի անունն է, return type չունի:
2. Ոչ, return type չունի (նույնիսկ void ոչ): Քանի որ նրա նպատակը object-ի կառուցումն է, ոչ արժեք վերադարձնելը; լեզուն այդպես է սահմանված:
3. Parameter չունեցող constructor; կանչվում է `A a;` դեպքում; եթե ոչ մի constructor չգրես, compiler-ը գեներացնում է implicit default constructor:
4. Constructor, որ ընդունում է parameter(ներ) և դրանցով initialize է անում member-ները, օր.՝  
`Person(string n): name(n){}` :
5. Նույն class-ում մի քանի constructor՝ տարբեր parameter list-երով; compiler-ը ընտրում է ըստ argument-ների (compile-time):
6. Init list-ը member-ները initialize է անում մարմնից առաջ՝ ուղղակի construction-ով, ոչ թե default-construct + assign; ավելի արդյունավետ է, և `const` /reference member-ների ու base class-ի համար պարտադիր է:
7. Ոչ: Virtual dispatch-ը հիմնվում է vptr-ի վրա, որը դեռ set չէ object-ի կառուցման պահին; բացի այդ object-ը դեռ գոյություն չունի, ուստի «որ տիպի» constructor-ը կանչել իմաստ չունի:
8. Այո: Օգտագործվում է, երբ ուզում ես կառավարել object-ի ստեղծումը՝ Singleton, factory method, կամ արգելել ուղղակի instantiation:
9. Singleton-ը երաշխավորում է class-ի միայն մեկ instance; constructor-ը private է, որ դրսից object չստեղծվի, և `static getInstance()` -ը վերադարձնում է միակ static instance-ը (`static T instance;`):
10. Destructor-ը հատուկ member function է (`~Class()`), որ ավտոմատ կանչվում է object-ի ոչնչացման ժամանակ; նպատակը՝ resource cleanup (memory, file, socket):
11. Stack object -> scope-ից դուրս գալիս ավտոմատ; heap object (`new`) -> միայն `delete` -ի ժամանակ:
12. Որ object-ի բռնած resource-ները ազատվեն (RAII): heap memory, file handle, DB connection, lock; հակառակ դեպքում leak կամ resource exhaustion:
13. Ոչ մեկը. destructor-ը parameter չունի (ուստի overloading հնարավոր չէ) և return type չունի:
14. Ոչ, միայն մեկ destructor:
15. LIFO — վերջին կառուցված object-ն առաջինը ոչնչանում; `Test a; Test b;` -> ~b հետո ~a:

**16.** Base destructor-ը virtual դարձնելը երաշխավորում է, որ base pointer-ով delete անելիս derived destructor-ն էլ կանչվի; կարևոր է polymorphic base class-ների համար՝ leak-ից խուսափելու:

**17.** Կանչվում է միայն base destructor-ը (static binding), derived-ի cleanup-ը չի կատարվում -> resource/memory leak, undefined behavior:

**18.** Special constructor, որ նոր object է ստեղծում գոյություն ունեցող object-ի հիման վրա.

`A(const A& other) :`

**19.** (1) `A b = a;` / `A b(a);` initialization; (2) function-ին by value փոխանցում; (3) function-ից by value return (հաճախ optimize է լինում):

**20.** Shallow copy — պատճենում է միայն pointer-ի արժեքը (հասցեն); երկու object նույն memory-ն են ցույց տալիս -> shared state, double delete վտանգ:

**21.** Deep copy — նոր memory է allocate անում և պատճենում իրական բովանդակությունը; object-ները անկախ են:

**22.** Որ (1) copy չլինի (reference), և հատկապես որ (2) infinite recursion-ից խուսափի — by value parameter-ը ինքը copy constructor կպահանջեր; `const` -ը երաշխավորում է original-ը չփոխվի:

**23.** Նույն pointer-ը երկու անգամ delete անելը (shallow copy-ից հետո երկու object նույն memory-ն են ազատում) -> undefined behavior, crash/heap corruption:

**24.** Move constructor-ը resource-ի ownership է փոխանցում (pointer «գողանում»), նոր allocation/copy չի անում; դրա համար մեծ object-ների համար շատ ավելի արագ է քան deep copy:

**25.** rvalue reference ( `T&&` ) bind է լինում rvalue-ների (temporary-ների) հետ; move semantics-ի հիմքն է. թույլ է տալիս «գողանալ» temporary-ի resource-ները:

**26.** Ոչինչ չի տեղափոխում. պարզապես `static_cast<T&&>` է անում, որ object-ը treat լինի որպես rvalue և move constructor/assignment-ը ընտրվի:

**27.** lvalue — ունի անուն ու հասցե, կարող ես `&` -ով վերցնել (օր. `x` , `arr[0]` ); rvalue — temporary արժեք, address չունի (օր. `5` , `x+1` , `foo()` ):

**28.** Քանի որ ցանկացած named variable lvalue է, նույնիսկ եթե տիպը rvalue reference է; `r` -ն անուն ունի, ուստի օգտագործելիս lvalue է: Move անելու համար պետք է `std::move(r)` :

**29.** Rule of Zero — եթե օգտագործես RAII տիպեր ( `std::string` , `std::vector` , `std::unique_ptr` ), ընդհանրապես ձեռքով copy/move/destructor գրելու կարիք չկա; compiler-ի default-երը ճիշտ են աշխատում:

**30.** `new int` — uninitialized (աղբ արժեք, UB կարդալիս); `new int()` — value-initialized 0-ով; `new int{}` (C++11) նույնպես 0:

**31.** Object, որ compiler-ը ստեղծում է կարճ ժամանակով (օր. `std::string("x")` , `a+b` ); սովորաբար expression-ի վերջում ոչնչանում է (եթե const ref-ը չերկարացնի lifetime-ը):

**32.** Non-owning view, որ պահում է միայն pointer + length; ընթերցման համար string-ի copy/allocation չի անում, ուստի function parameter-ի համար օպտիմալ է, երբ միայն կարդում ես: Զգույշ dangling-ից:

**33.** RVO/NRVO — compiler optimization-ներ, որ վերացնում են return value-ի copy/move-ը՝ object-ն ուղղակի կառուցելով վերջնական տեղում; C++17-ում RVO-ն (temporary-ի դեպքում) երաշխավորված է:

**34.** Small String Optimization — փոքր string-երը պահվում են հենց `std::string` object-ի ներսում (buffer), առանց heap allocation; ուստի կարճ string-երը շատ արագ են:

**35.** `const` -ը երաշխավորում է, որ object-ը initialization-ից հետո չի փոփոխվի; օգնում է անվտանգության և optimization-ի; lifetime-ը չի փոխում:

**36.** `const int* p` — տվյալն է `const` (`*p` -ն չես կարող փոխել, `p` -ն կարող ես); `int* const p` — pointer-ն է `const` (`p` -ն չես կարող փոխել, `*p` -ն կարող ես): Կարդա աջից ձախ:

**37.** `T f() const;` — երաշխավորում է, որ function-ը չի փոփոխի object-ի (non-mutable) member-ները; `this` -ը դառնում է `const`; `const` object-ների վրա միայն `const` method կարելի է կանչել:

**38.** (1) Copy չի լինում (reference, մեծ object-ի համար էժան); (2) function-ը չի կարող փոխել original-ը (`const`): Բռնու՝ ընդունում է և lvalue, և temporary:

**39.** Variable, որ ստեղծվում է առաջին անգամ function կանչվելիս և ապրում մինչև program end՝ պահպանելով արժեքը կանչերի միջև; initialization-ը կատարվում է մեկ անգամ (thread-safe C++11-ից):

**40.** Class-ին (ոչ object-ին) — մեկ shared instance բոլոր object-ների համար; define-ը class-ից դուրս՝ `int A::count = 0;`; հասանելի է `A::count` -ով:

**41.** Քանի որ static function-ը object-ի հետ կապ չունի, չունի `this` pointer; non-static member-ը կոնկրետ object է պահանջում, ուստի static function-ից անհասանելի է:

**42.** `const` — immutable object (կարող է runtime-ում initialize լինել); `constexpr` — compile-time constant/function (արժեքը հայտնի է compile time-ում): Ամեն `constexpr const` է, բայց ոչ հակառակը:

**43.** Virtual function-ը թույլ է տալիս call-ը resolve լինել runtime-ում ըստ object-ի իրական տիպի (ոչ pointer-ի static տիպի); սա runtime polymorphism-ն է՝ նույն interface, տարբեր behavior:

**44.** Static (early) binding — call-ը որոշվում է compile-time-ում (non-virtual function); dynamic (late) binding — runtime-ում vptr/vtable-ով (virtual function):

**45.** Override — derived class-ը վերասահմանում է base-ի virtual function-ը նույն signature-ով (runtime, inheritance պետք է); Overload — նույն անունով function-ներ տարբեր parameter-ներով (compile-time, inheritance պետք չէ): Signature-ի տարբերությունը override-ը վերածում է name hiding-ի:

**46.** vtable — per-class աղյուսակ virtual function-ների հասցեներով (class level, մեկ հատ); vptr — per-object hidden pointer, որ ցույց է տալիս class-ի vtable-ին (object level):

**47.** (1) object-ից կարդում vptr; (2) vptr-ով մտնում vtable; (3) գտնում function-ի slot-ի հասցեն; (4) jump/call այդ հասցեին -> իրական (derived) implementation:

**48.** Pure virtual function ( = 0 ) — implementation չունեցող virtual, որ derived-ը պարտավոր է override անել; class-ը, որ ունի գոնե մեկ pure virtual, abstract է -> object չի ստեղծվում, բայց pointer/reference կարելի է:

**49.** `static_cast` — ստուգված compile-time փոխակերպումներ (թվային, upcast, void\*);

`dynamic_cast` — runtime-ստուգված downcast polymorphic տիպերի; `const_cast` — const/volatile ավելացնել/հեռացնել; `reinterpret_cast` — bit-level, ամենավտանգավոր: Խուսափիր C-style cast-ից:

**50.** static\_cast downcast-ը կոմպիլվում է բայց runtime-ում չի ստուգում -> սխալ տիպի դեպքում UB; dynamic\_cast-ը ստուգում է RTTI-ով -> ձախողման դեպքում nullptr (pointer) կամ bad\_cast (reference)՝ ապահով:

---

# ANSWER KEY C (Tricky / Overload / UB)

C1-a · C2-c · C3-b · C4-c · C5-c · C6-b · C7-a · C8-a · C9-c · C10-a C11-b · C12-a · C13-b · C14-c · C15-b · C16-c · C17-b · C18-a · C19-b · C20-b C21-c · C22-c · C23-a · C24-b · C25-b · C26-c · C27-a · C28-b · C29-a · C30-b C31-b · C32-b · C33-b · C34-b · C35-b · C36-b · C37-b · C38-b · C39-b · C40-a C41-a · C42-b · C43-a · C44-b · C45-b · C46-b · C47-b · C48-a · C49-a · C50-b C51-b · C52-b · C53-b · C54-b · C55-b · C56-b · C57-a · C58-a · C59-b · C60-b C61-a · C62-b · C63-a · C64-b · C65-b · C66-b · C67-b · C68-b · C69-b · C70-b C71-b · C72-b · C73-b · C74-b · C75-a · C76-b · C77-b · C78-b · C79-b · C80-b C81-b · C82-b · C83-b · C84-b · C85-b · C86-b · C87-b · C88-b · C89-b · C90-b C91-b · C92-a · C93-b · C94-b · C95-a · C96-a · C97-b · C98-b · C99-a · C100-b

## Մանրամասն բացատրություններ (C)

**C1** lvalue `x` -> `int&` ընդամենը ճշգրիտ match; -> lref. **C2** `5` rvalue -> `int&&` preferred over `const int&` -> rref. **C3** `const int` lvalue -> `int&` չի ընդունում, `int&&` ոչ -> `const int&` -> clref. **C4** `std::move(x)` xvalue (rvalue) -> `int&&` -> rref. **C5** `x+1` prvalue -> rref. **C6** `const xvalue` -> `int&&` չի ընդունում `const`, `const int&` է -> clref (այս set-ում `crref` չկա). **C7** rvalue `5` -> `const int&` (միակ որ ընդունում rvalue) -> clref. **C8** lvalue -> `int&` preferred over `const int&` -> lref. **C9** `f(int)` և `f(int&)` lvalue-ի համար **ambiguous**... բայց ստանդարտում lvalue-ի դեպքում `int&` exact ref match, `int` copy — իրականում սա **ambiguous** է շատ compiler-ներում? Ճիշտ պատասխան. lvalue-ի դեպք՝ երկուսն էլ viable, ranking հավասար -> **ambiguous CE**. Ուշադրություն. այս հատուկ դեպքում պատասխանը (c) CE է. (Տես ուղղում ներքևում.) **C10** `5` rvalue -> `int&` չի ընդունում -> `f(int)` -> val. **C11** `5` rvalue -> `int&&` preferred over `int(val)` for rvalue -> rref. **C12** lvalue -> `int&&` չի ընդունում -> `f(int)` -> val. **C13** `5` rvalue -> `const int&&` (rvalue-ին ավելի ճշգրիտ) preferred over `const int&` -> ... ուշադրություն. rvalue-ի համար `const int&&` ավելի լավն է քան `const int&` -> **crref**. Ուղղում. C13 = **b (crref)**. **C14** lvalue -> `const int&&` չի ընդունում lvalue -> `const int&` -> clref. **C15** `std::move(x)` rvalue -> `const int&&` ? Ոչ, non-const rvalue -> `const int&&` viable, `const int&` viable; rvalue ref preferred -> **crref**. **C16** `const lvalue` -> ոչ `int&` (const), ոչ `int&&` (lvalue) -> **CE**. **C17** rvalue -> `int&&` -> rref. **C18** `'A'` char literal -> `f(char)` exact -> char. **C19** `true` bool -> `f(int)` (bool->int promotion; char-ի փոխարկումը conversion է, promotion preferred) -> int. **C20** `float` -> `double` (floating promotion) preferred over int -> double. **C21** `int` -> `long` և `double` երկուսն էլ conversion; integer->long integral conversion vs int->double floating; **ambiguous**? իրականում int->long integral conversion, int->double floating conversion — երկուսն էլ "conversion" նույն rank -> **ambiguous CE**. Ուղղում. C21 = **c**. **C22** `int` -> `unsigned` և `long` երկուսն էլ integral conversion նույն rank -> ambiguous CE. **C23** `0` -> int exact match preferred over pointer (0->ptr conversion) -> int. **C24** `nullptr` -> `char*` (`nullptr_t` -> pointer) preferred over int -> ptr. **C25** `"hello"` -> `const char*` exact-ish (array->ptr) preferred over user-conversion to string -> cstr. Ուղղում. C25 = **b (cstr)**. **C26** `int&` չի ընդունում `const lvalue` ->

CE. **C27** non-const object -> non-const `g()` -> nc. **C28** const object -> const `g()` -> c. **C29** lvalue object -> `g() &` -> lval. **C30** `S{}` rvalue -> `g() &&` -> rval. **C31** C++17. argument evaluation-ի order-ը unspecified, բայց indeterminately sequenced (նշ UB). Մինչ C++17' UB. **C32** C++17-ում `i = i++` defined -> `i==0` (RHS-ի `i++` արժեքը 0, հետո assign 0, ++ side effect sequenced before assignment). **C33** `++i + i++` — i-ի two unsequenced modifications -> UB (C++17-ում էլ). **C34** C++17. assignment-ի RHS sequenced before LHS -> defined. **C35** `x++ + ++x` — unsequenced modifications of x -> UB. **C36** `Base b = d;` object slicing -> b-ն միայն Base մասն է -> B. **C37** `Base& b = d;` reference, virtual -> D. **C38** `s` const -> move constructor (`string&&`) չի ընդունում const -> copy constructor -> copy. **C39** `r` named rvalue reference -> lvalue -> `f(int&&)` -ին չի bind -> CE. **C40** `std::move(r)` -> rvalue -> rref. **C41** rvalue `5` -> `int&&` (non-const rvalue exact) -> rref. **C42** const rvalue -> `const int&&` -> crref. **C43** lvalue -> `int&` -> lref. **C44** forwarding reference `T&&` lvalue-ի դեպքում `T=int&`, bind -> universal. **C45** `"hello"` -> `const char*` -> string (user conversion) vs bool. `const char*` -> bool boolean conversion, `const char*` -> string user-defined. Standard conversion (to bool) rank-ը... իրականում array->pointer->bool երկուսն էլ. Ստանդարտ. pointer->bool boolean conversion vs user-defined to string. Boolean conversion-ը standard է, user-defined-ից preferred -> **bool?! Ուղղում.** C45 = **b (bool)** — famous trap. (Տես ուղղում.) **C46** `5L` long -> `f(long)` exact -> long. **C47** `short` -> `f(short)` exact -> short. **C48** `5` int literal -> `f(int)` exact -> int. **C49** unscoped enum -> int conversion -> int. **C50** scoped enum -> no implicit conversion to int -> CE. **C51** `NULL` սովորաբար `0` (int) -> `f(int)` -> int (famous trap; նախընտրիր nullptr). **C52** `5` int -> `f(double)` standard conversion preferred over user-defined `A(int)` -> double. **C53** նույն signature -> redefinition -> CE. **C54** return type-ը overload չի սահմանում -> CE. **C55** `f(3)` — `f(int)` և `f(int, int=5)` երկուսն էլ viable -> ambiguous CE. **C56** `std::move(a)` -> move constructor -> "ctor move". **C57** `A b = a;` lvalue -> copy -> "ctor copy". **C58** C++17 RVO/NRVO -> միայն "ctor". **C59** `std::move(a)` -> string move constructor -> move. **C60** `auto&&` + lvalue -> `int&`. **C61** `auto&&` + rvalue -> `int&&`. **C62** `auto` drops top-level const -> y is `int`, mutable -> OK. **C63** `sizeof(arr)/sizeof(arr[0])` = 20/4 = 5. **C64** parameter `int arr[5]` decays to `int*` -> sizeof = 8 (64-bit). **C65** `A a;` POD member uninitialized -> undefined. **C66** reference chain -> r2 aliases x -> x==20. **C67** returning reference to local -> dangling -> UB. **C68** const ref extends temporary lifetime -> 5. **C69** rvalue ref is modifiable lvalue after binding -> 10. **C70** `for(auto x: v)` copy -> v unchanged -> 1. **C71** `for(auto& x: v)` reference -> v[0]=2. **C72** non-virtual dtor + delete base ptr -> only ~Base -> "B" + leak. **C73** int+double -> double -> 8.9. **C74** unsigned vs signed comparison -> -1 converts to large unsigned -> a>b false? -1 -> UINT\_MAX, so a(1) > b(UINT\_MAX) is false -> 0... Ուղղում. C74 = **b (0)**. (a=1, b becomes huge, 1 > huge = false = 0.) **C75** char prints 'A', (int)c prints 65. **C76** `if(x=0)` assignment -> 0 -> false -> no. **C77** C++17. `cout << i++ << i` -> left-to-right sequenced -> 56. **C78** unique\_ptr copy deleted -> CE. **C79** move -> OK, p1 becomes nullptr. **C80** shared\_ptr copy -> use\_count==2. **C81** noexcept function throws -> std::terminate. **C82** integer division by zero -> UB. **C83** floating division by zero -> inf (not UB on IEEE platforms). **C84** `v[10]` out of bounds -> UB. **C85** `v.at(10)` -> throw out\_of\_range. **C86** `s[10]` out of range -> UB. **C87** null dereference -> UB. **C88** double delete -> UB. **C89** use after free -> UB. **C90** virtual call in constructor -> resolves to A -> "A". **C91** virtual override -> B. **C92** non-virtual -> static binding ->



A. **C93** switch fall-through (no break after case 3) -> "34". **C94** `for(...);` empty body -> "hi" once. **C95** nested ternary: `x>3 true -> x<10 true -> 1`. **C96** constexpr array size -> 9. **C97** const ref alias -> `x=10` reflects -> 10. **C98** `int&` չի bind rvalue -> `f(int)` -> val. **C99** `A a{}` value-init -> `x==0`. **C100** `A a;` default-init POD -> undefined.

## Կարևոր ուղղումներ / ճշգրտումներ (կարդա անպայման)

Overload resolution-ը շատ նուրբ է; մի քանի հարց «trap» է հենց այն պատճառով, որ ինտուիցիան սխալ է տանում.

- **C9:** lvalue-ի դեպքում `f(int)` (by value) և `f(int&)` երկուսն էլ exact-rank viable են -> իրականում **ambiguous (CE)**. Answer key-ի տառը (b) պարզեցված է; խիստ ստանդարտով սա ambiguous է: Հարցադրույցին ասա՝ «`int` և `int&` overload-ը lvalue-ի համար ambiguous է»:
- **C13:** rvalue `5` -ի համար `const int&&` ավելի ճշգրիտ է քան `const int&` -> **crref (b)**:
- **C21 / C22:** `int` -ից `long` / `double` / `unsigned` — մի քանի հավասար-rank conversion -> **ambiguous (CE)** (C21=c, C22=c):
- **C25:** `"hello"` (`const char[6]`) -> `const char*` array-to-pointer (Exact-ish rank) preferred over user-defined `string` conversion -> **cstring (b)**:
- **C45:** `const char*` -> `bool` boolean conversion-ը standard conversion է և preferred է user-defined `string` conversion-ից -> **bool (b)** (հայտնի «bool overload» trap):
- **C74:** signed/unsigned համեմատում. `-1` -> շատ մեծ unsigned, ուստի `1 > huge` -> **false = 0 (b)**:

Ընդհանուր կանոններ overload resolution-ի համար (հիշիր հարցադրույցին).

1. Exact match > Promotion > Standard conversion > User-defined conversion >
2. rvalue -> preferred: `T&&` > `const T&&` > `const T&` (`T&` չի ընդունում rvalue)
3. lvalue -> preferred: `T&` > `const T&` (`T&&` չի ընդունում lvalue)
4. `const lvalue` -> միայն `const T&` (կամ `const T&&`-ը ոչ) -> հաճախ միայն `const T&`
5. Named rvalue reference-ը (`int&& r`) օգտագործելիս LVALUE է
6. Two equally-ranked viable functions -> ambiguous (compile error)
7. Return type-ը Չի մասնակցում overload resolution-ին

Sequence points / UB կանոններ (C++17).

- նույն scalar object-ի երկու չ-sequenced modification նույն expression-ում -> UB (օր. ++i + i++, x++ + ++x)
- i = i++ -> C++17-ում defined (i unchanged)
- a[i] = i++ -> C++17-ում defined
- foo(x++, x++) -> argument order unspecified (ՈՉ UB C++17-ում), UB մինչ C++17
- cout << i++ << i -> C++17-ում left-to-right sequenced (56 for i=5)
- integer / 0 -> UB; double / 0 -> inf (IEEE)
- out-of-bounds [], null deref, double delete, use-after-free, dangling ref -> UB